Table des matières

Cours 11 — Qualité, contrats et observabilité 1
Objectifs 2
1. Un pipeline qui ne plante pas peut mentir 3
2. Le contrat de données 3
3. Les dimensions d'une règle 4
4. Bloquer, avertir ou suivre 6
5. Un contrat a une version 6
6. Le contrôle de distribution 7
7. Alerte utile, alerte ignorée 8
8. L'observabilité : lire une plateforme par ses traces 9
9. Conduire un incident de données 10
10. Les outils : ce que le marché ajoute 10
Points clés à retenir 12
Exercices 12

Cours 11 — Qualité, contrats et observabilité

Théorie · Module 11/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → **ingestion** → **lac** → **entrepôt** → restitution. Les dix premiers cours ont appris à faire arriver la donnée, à la ranger et à la faire tourner à l'heure. Celui-ci apprend à prouver qu'elle est bonne, à chaque livraison, et à dire à qui il faut quand elle ne l'est pas. Il ne déplace aucune donnée : il la juge, là où les autres cours l'ont posée.

En 30 secondes :

- Un pipeline se trompe rarement en plantant. Le mois d'avril 2026, qui n'existe pas dans les sources, se charge sans erreur : zéro ligne, code de sortie 0, et Airflow passe au vert.
- Un contrat de données dit, par écrit, ce que le producteur promet et ce que le consommateur peut tenir pour vrai. Chez Boréal, il vit dans un fichier YAML du dépôt, et chaque règle y porte sa métrique, son attendu et sa sévérité.

© 2026 iAlexMG. Tous droits réservés.

- Bloquer, avertir ou suivre ne se décide pas une fois pour toutes : l'extrait absent bloque, la commande relivrée avertit, la coupure de communication se suit.
- Un contrat a une version. Le micrologiciel 2.0 a renommé un champ le 2025-05-01 ; le contrat 1.0.0 bloque ce jour-là, et un consommateur qui n'en avait pas a perdu la température de 100 % des messages dès août 2025, juste avant les surchauffes de l'hiver.
- Un contrôle de distribution voit la pression instable du réseau d'air dès le 20 octobre 2025, huit jours avant le déclenchement du compresseur 2 : 7 alertes, toutes pendant la dérive, 0 fausse. Écrit sans soin, le même contrôle en donne 8 de plus, toutes fausses.
- Une alerte qui sonne un jour sur trois n'est plus lue. Les contrats se rejouent sur l'historique avant de sonner pour de vrai, et chaque alerte se compte : vraie, fausse, ou manquée.

L'idée du cours. Un test vérifie le code ; un contrat vérifie la donnée. Le code du pipeline ne change pas d'une nuit à l'autre, la donnée si : un fournisseur livre deux fois le même mois, un micrologiciel renomme un champ, un compresseur s'use. Aucun test unitaire ne voit ces défauts, parce qu'ils ne sont pas dans le code. Ils sont dans ce qui arrive, et c'est là qu'il faut regarder, à chaque fois.

Le cours compte deux séances. La première, sections 1 à 5 et 8, tient les contrats de ce que la `RAW` reçoit : l'ERP et la télémétrie. La seconde, sections 6, 7 et 9, tient l'usine, les alertes qu'on garde et celles qu'on retire, les contrats dans les DAG, et l'incident de données.

Objectifs

- Dire pourquoi une donnée fausse coûte plus cher qu'un pipeline arrêté, et où la valider.
- Décrire un contrat de données : son produit, son producteur, ses consommateurs, sa version et ses règles.
- Classer une règle par sa dimension : fraîcheur, volume, unicité, validité, complétude, intégrité, cohérence, schéma, distribution.
- Choisir, règle par règle, entre bloquer, avertir et suivre, selon ce que coûte l'erreur en aval.
- Faire évoluer un contrat par sa version, et reconnaître un changement qui casse.
- Bâtir un contrôle de distribution sur la vraie sortie, et en compter les fausses alertes.
- Rejouer un contrat sur l'historique pour juger ses alertes avant qu'elles sonnent.
- Lire l'état d'une plateforme par ses traces : chargements, exécutions, verdicts, lignage.
- Conduire un incident de données, du signal au post-mortem.
- Situer le contrat parmi les outils du marché et les tests d'Analytics Engineering.

© 2026 iAlexMG. Tous droits réservés.

1. Un pipeline qui ne plante pas peut mentir

Labo : `Labo_11_DE.md`, étape 1.

Depuis le Cours 05, un pipeline qui échoue le dit : un code de sortie, une ligne « échoué » dans `pipeline.executions`, une tâche rouge dans Airflow. C'est la panne la moins grave. La plus grave est celle qui ne plante pas, et Boréal en a déjà montré trois.

Défaut	Ce que le pipeline en dit	Ce qui se passe en aval
L'extrait de mars 2023 livré deux fois (Cours 03)	rien : un autre nom de dossier, donc une autre livraison	609 commandes comptées deux fois par qui somme la `RAW` sans dédoublonner
Le mois d'avril 2026 demandé à la `RAW`	« à jour », zéro ligne, code 0	un DAG mensuel au vert pour un mois qui n'est jamais arrivé
Le champ renommé du micrologiciel 2.0	rien : le message est du JSON, il entre tel quel	un consommateur qui lit `temp_moteur_max` voit sa part de messages remplis tomber à 0 % en août 2025

La troisième ligne se mesure au labo, mois par mois. En avril 2025, 100 % des messages portent `temp_moteur_max` ; en mai, 77,8 % ; en juillet, 4,2 % ; à partir d'août, plus aucun. Un tableau de bord de la température du moteur, écrit en 2024 sur ce champ, devient muet sans une erreur. Et l'hiver qui suit est celui des surchauffes : la température réelle monte à 120,7 °C en février 2026, dans un champ que ce tableau de bord ne lit pas.

Le coût d'une donnée fausse croît avec la distance qu'elle parcourt. Arrêtée à l'entrée, elle coûte une relance ; passée dans la `RAW`, puis dans le `STAGING`, puis dans un tableau de bord, elle coûte une décision prise sur un faux chiffre, et la confiance de ceux qui l'ont prise. D'où la règle du cours : valider à l'entrée, là où l'on sait encore d'où vient la donnée, plutôt que déboguer en aval, là où on ne le sait plus.

2. Le contrat de données

Labo : `Labo_11_DE.md`, étapes 2 et 9.

Un contrat de données (*data contract*) est l'accord écrit entre celui qui produit une donnée et ceux qui la consomment. Il dit ce qui est livré, sous quelle forme, avec quelles garanties, et ce qui se passe quand une garantie tombe. Il n'est pas un document qu'on range : il s'exécute, à chaque livraison, et son verdict s'enregistre.

Ce que porte un contrat	Chez Boréal
Le produit	`raw.erp_vbak` et `raw.erp_kna1`, les extraits de l'ERP tels que la `RAW` les garde
Le producteur	l'équipe de l'ERP, qui dépose un extrait par mois

© 2026 iAlexMG. Tous droits réservés.

Ce que porte un contrat	Chez Boréal
Les consommateurs	Analytics Engineering, par son `STAGING`
La version	`1.0.0`, puis la suivante quand une règle change
La fenêtre	la colonne qui situe une ligne dans le temps : `_lot`, ou `jour`
Les règles	une métrique, un attendu, une sévérité, et une dimension qui la classe

Le contrat vit dans un fichier, dans le dépôt, à côté du code qui l'exécute. C'est ce qui le rend discutable : le producteur le lit, le consommateur le lit, et une règle qui change laisse une trace dans Git, avec son auteur et sa raison. Au labo, `01_de/contrats/` en tient quatre, un par produit : les extraits de l'ERP, la télémétrie, l'historien de l'usine, et les séries de l'argent.

Les trois premiers jugent ce que Boréal reçoit : Data Engineering y est le consommateur. Le quatrième juge ce que Data Engineering livre : cette fois, c'est lui le producteur, et les consommateurs s'appellent Analytics Engineering, Data Science et Business Intelligence. La même équipe se tient ainsi des deux côtés d'un contrat, selon le produit, et c'est la situation normale d'un ingénieur de données.

Un standard ouvert décrit ces fichiers : l'ODCS (*Open Data Contract Standard*), tenu par le projet Bitol de la Linux Foundation, en version 3.2.0 depuis septembre 2026 d'après son dépôt, consulté le 2026-09-23. Il couvre le schéma, les règles de qualité, les engagements de service, les propriétaires et les serveurs. Ses règles de qualité ont la forme de celles du labo : une métrique d'une petite bibliothèque, ou une requête SQL, comparée à un attendu.

```yaml
schema:
  - name: orders
    quality:
      - id: orders_unique_tenant_order
        metric: duplicateValues
        mustBe: 0
        arguments:
          properties:
            - tenant_id
            - order_id
```

Les contrats du labo suivent cette forme en français, en plus petit : `metrique: doublons` pour `duplicateValues`, `doit_etre` pour `mustBe`, `au_plus` et `au_moins` pour ses bornes. Un petit module du pipeline, `pipeline/contrats.py`, traduit chaque règle en une requête DuckDB qui rend un seul nombre. La mécanique se voit en entier : c'est le choix du parcours, et la section 10 dit ce que les outils du marché ajoutent.

3. Les dimensions d'une règle

Labo : `Labo_11_DE.md`, étapes 2, 3 et 6.

© 2026 iAlexMG. Tous droits réservés.

Une règle vérifie une chose, et cette chose se range dans une des familles que le métier nomme dimensions de la qualité. Les nommer sert à voir ce qui manque : un contrat qui n'a que des règles de validité ne dira jamais qu'un mois n'est pas arrivé.

Dimension	La question	Chez Boréal	La métrique du labo
Fraîcheur	ce qui devait arriver est-il arrivé ?	l'extrait du mois, l'archive du jour, l'export de l'historien	`periodes_manquantes`
Volume	en est-il arrivé assez, ou trop ?	au moins 300 commandes par extrait, le plus petit en comptant 380	`sql`
Unicité	chaque fait est-il là une fois ?	une commande par lot ; une commande relivrée à l'identique	`doublons`, `sql`
Validité	chaque valeur est-elle possible ?	des numéros SAP à dix chiffres ; une mesure dans la plage de son capteur	`invalides`
Complétude	chaque ligne porte-t-elle ce qu'elle doit porter ?	le point sans valeur d'une coupure de communication ; l'heure couverte sans moyenne	`lignes`, `invalides`
Intégrité	ce qui est cité existe-t-il ?	le concessionnaire d'une commande, l'étiquette d'un point	`invalides`, `sql`
Cohérence	deux chiffres qui doivent s'accorder s'accordent-ils ?	la production d'une journée contre le cumul du compteur	`sql`
Schéma	la forme est-elle celle qu'on attend ?	les champs du message de télémétrie	`champs_json`
Distribution	les valeurs ressemblent-elles à d'habitude ?	la pression du réseau d'air pendant la marche des lignes	`sql`

Les questions de la deuxième colonne sont celles qu'un analyste pose ; la règle les rend vérifiables par une machine. La fraîcheur et le volume se jugent sur la livraison entière, sans ouvrir une ligne ; l'unicité, la validité, la complétude et l'intégrité, ligne par ligne ; la cohérence, entre deux tables ou deux calculs qui disent la même chose ; la distribution, sur l'ensemble des lignes, contre ce qu'on a vu avant. Cette dernière est la seule qui ait besoin de mémoire, et la section 6 lui est consacrée.

Une règle de fraîcheur demande une précaution. Elle doit savoir ce qui était attendu, et pas seulement ce qui est là : une table ne peut pas dire qu'il lui manque une ligne qu'elle n'a jamais reçue. Au labo, `periodes_manquantes` compare les mois ou les jours présents à ceux de la fenêtre de l'exécution, et c'est ce qui lui fait voir le mois d'avril demandé et absent. Pour la télémétrie, elle lit le journal des chargements plutôt que la table : 41 archives de 2023 sont vides, dont les 39 premières, et une archive vide est une archive livrée.

© 2026 iAlexMG. Tous droits réservés.

4. Bloquer, avertir ou suivre

Labo : `Labo_11_DE.md`, étapes 3, 8 et 10.

Une règle rompue déclenche quelque chose, et ce quelque chose se décide règle par règle. Le labo en offre trois degrés.

Sévérité	Ce qu'elle fait	Quand la choisir	Chez Boréal
bloquant	l'étape échoue, la tâche du DAG passe au rouge, rien ne se construit en aval	quand tout ce qui suit serait faux	l'extrait absent, les numéros mal formés, un champ du message inconnu
avertissement	l'étape passe, et une alerte s'écrit, pour quelqu'un qui doit regarder	quand l'aval sait traiter le défaut, mais qu'une personne doit le savoir	la commande relivrée, le concessionnaire en retard, la pression instable
suivi	l'étape passe, et le verdict s'inscrit dans la table, sans alerte	quand le défaut est connu, compté, et qu'aucune action n'est attendue ce jour-là	les points douteux, les coupures de communication

La question qui tranche est celle du coût en aval. Un extrait absent casse tout ce qui suit : on bloque. Une commande relivrée à l'identique, la `RAW` la garde de toute façon, marquée de son lot (D60), et le `STAGING` d'Analytics Engineering sait n'en retenir qu'une : bloquer le mois entier pour elle ferait plus de dégâts que le défaut. On avertit. Le concessionnaire créé le dernier jour de mai 2024 arrive au lot de juin : c'est un défaut qui se résout seul, et on avertit aussi.

Bloquer ne veut pas dire refuser la donnée. Au labo, les contrats tournent après le chargement de la `RAW`, qui garde tout ce qui est arrivé ; ce qu'une règle bloquante arrête, c'est ce qui se construirait dessus, la tâche `dbt_build` des DAG. Une donnée refusée à l'entrée serait perdue, et la `RAW` cesserait d'être le brut. Valider à l'entrée veut dire juger dès l'entrée, pas jeter.

Le module juge toutes les règles avant de décider. Une exécution qui s'arrêterait à la première règle rompue cacherait les suivantes, et on corrigerait un défaut à la fois, une relance à la fois.

Pourquoi — la fenêtre, jusque dans les contrats. Un contrat jugé sur tout l'historique sonne à chaque passage pour l'extrait de mars 2023, livré deux fois il y a trois ans et connu de tous. Jugé sur la fenêtre de l'exécution, celle du Cours 06, il sonne une fois : le jour où ce mois arrive. Au labo, le DAG mensuel passe la fenêtre de son mois aux contrats, et le quotidien celle de sa journée.

5. Un contrat a une version

Labo : `Labo_11_DE.md`, étape 4.

Le contrat de la télémétrie a commencé en version 1.0.0, avec une règle de schéma : chaque message porte exactement les treize champs des micrologiciels 1.x. Le 2025-05-01, quatre messages arrivent avec `temperature_moteur_max` au lieu de `temp_moteur_max`, et un quatorzième champ, `heures_moteur`. La règle

© 2026 iAlexMG. Tous droits réservés.

rompt, et elle est bloquante : le pipeline s'arrête le jour même, avec quatre messages hors contrat. Sans contrat, rien ne s'arrête, et le renommage se découvre des mois plus tard, dans un tableau de bord muet (section 1).

Un contrat rompu ne se contourne pas en baissant la sévérité de la règle. Il se renégocie. Le producteur, l'équipe du module télématique, confirme que le 2.0 est le nouveau format ; les consommateurs disent ce qu'il leur faut pour lire les deux ; et le contrat passe en version 2.0.0, qui déclare les deux formes du message. La règle de schéma juge alors 115 807 messages 2.0 comme conformes, et 0 hors contrat.

Changement	Exemple	Ce qu'il fait au consommateur	La version
Qui casse	un champ renommé, supprimé, ou dont le type change	sa requête rend `NULL` ou plante	premier chiffre : 1.0.0 → 2.0.0
Qui ajoute	un champ neuf, une valeur de plus dans une liste	rien, s'il ignore ce qu'il ne connaît pas	deuxième chiffre : 1.0.0 → 1.1.0
Qui corrige	une description, une règle mieux écrite	rien	troisième chiffre : 1.0.0 → 1.0.1

C'est la numérotation sémantique des bibliothèques de code, appliquée à une interface de données : le premier chiffre annonce qu'il faudra toucher au code qui lit. Le Cours 10 a montré comment un format de table encaisse un renommage par identifiant de colonne ; le contrat, lui, fait en sorte que le renommage soit décidé et annoncé avant d'arriver, et non découvert après.

Piège — accepter les deux formes pour toujours. La version 2.0.0 déclare les deux formes parce que des véhicules en 1.3 et en 1.6 ont publié sous l'ancien nom jusqu'au 2025-07-14. Une fois la flotte passée en 2.0, une version 3.0.0 pourra retirer la forme 1.x : un contrat qui n'oublie jamais rien finit par tout accepter.

6. Le contrôle de distribution

Labo : `Labo_11_DE.md`, étape 7.

Les règles des sections précédentes ont un attendu fixe : zéro doublon, au moins 300 commandes. Une règle de distribution compare une livraison à ce qu'on a vu avant. Elle répond à une question qu'aucune règle fixe ne pose : cette journée ressemble-t-elle aux autres ?

Le cas de Boréal est la pression au collecteur du réseau d'air, `UTL.AIR.PRESSION`, pendant la dérive du compresseur 2 (Cours 08). Jour par jour, sur les heures où les lignes tournent, son écart-type reste entre 2,0 et 6,3 kPa sur 114 des 121 journées de production ; hors de la seconde moitié d'octobre, sa médiane est de 4,0. Le 20 octobre 2025, il passe à 15,1, puis monte jusqu'à 60,4 le 28, jour du déclenchement. La règle du contrat juge chaque journée contre la médiane des vingt journées de production qui la précèdent, et alerte au-delà du double.

Version de la règle	Alertes pendant la dérive	Alertes ailleurs, fausses
l'écart-type de 7 h à 14 h, les jours de production	7	8

© 2026 iAlexMG. Tous droits réservés.

Version de la règle	Alertes pendant la dérive	Alertes ailleurs, fausses
l'écart-type pendant que les lignes tournent	7	0

Les huit fausses alertes de la première version tombent toutes sur des journées courtes, les 22 et 23 décembre et six jours de janvier : les lignes s'arrêtent vers 11 h, les compresseurs aussi, et la pression retombe à zéro pendant l'après-midi. Un écart-type énorme, pour une usine qui va très bien. C'est ce que veut dire bâtir le contrôle sur la vraie sortie : on écrit la règle, on la lance sur des mois de données, on regarde chaque alerte, et l'on comprend chaque fausse avant de corriger. Le filtre des lignes en marche vient de cet examen, pas d'une intuition.

Reste ce que la règle ne voit pas. La dérive commence le 15 octobre, et les journées des 15, 16 et 17 ne se distinguent pas encore par leur écart-type : trois jours manqués. Le nombre de points de pression, lui, double déjà le 17, 28 contre 12 d'habitude : la bande morte de l'historien écrit un point de plus chaque fois que la valeur bouge (Cours 08). Une règle sur le volume de points aurait alerté un jour ouvrable plus tôt, et c'est une question des exercices.

Un contrôle de distribution ne sait pas distinguer un défaut de la donnée d'un changement du monde. La pression instable d'octobre est une vraie mesure d'une vraie dérive : la donnée est juste, c'est le compresseur qui ne l'est pas. La règle avertit donc, et son alerte doit aller à qui peut agir, la maintenance de l'usine, et non à l'ingénieur de données qui n'y peut rien. La première alerte porte sur la journée du 20 octobre 2025 : huit jours avant le déclenchement, deux jours avant le bruit signalé par un opérateur le 22 et avant l'alerte de vibration du Cours 09.

7. Alerte utile, alerte ignorée

Labo : `Labo_11_DE.md`, étape 8.

Une alerte coûte l'attention de quelqu'un. Une alerte qui sonne pour rien use cette attention, et au bout de quelques semaines, plus personne ne lit les alertes, les bonnes comprises. Le métier appelle cela la fatigue d'alerte, et c'est la façon la plus courante de rendre un contrôle inutile.

Le seul remède est de compter. Avant de faire sonner un contrat, on le rejoue sur l'historique, tranche par tranche, comme le DAG l'aurait fait, et chaque alerte se classe.

Classe	Définition	Chez Boréal, l'historien rejoué sur 182 jours
Vraie	elle tombe pendant un événement réel	la pression instable, 7 jours ; le capteur figé, 1 jour
Fausse	elle tombe hors de tout événement	0, une fois la règle de pression corrigée
Manquée	un jour d'événement sans alerte	les 15, 16 et 17 octobre
Vraie mais inutile	l'événement est réel, mais personne n'a rien à faire ce jour-là	les points douteux, 55 jours ; les coupures, 44 jours

La dernière ligne est la plus instructive. Les 64 points douteux et les 529 points perdus aux coupures sont réels : l'historien les marque lui-même, et les séries de l'argent les comptent (Cours 08). Mais une alerte qui sonne un jour sur trois pour un défaut déjà traité ne mène à aucune action, et elle noie les deux alertes qui

© 2026 iAlexMG. Tous droits réservés.

comptaient. Passées en suivi, ces règles continuent de s'inscrire dans `pipeline.controles`, où un tableau de bord hebdomadaire les lira, et les alertes envoyées tombent de 107 à 8.

Trois questions jugent une alerte avant qu'elle sonne. Qui la reçoit ? Que fera cette personne ? Combien de fois par mois sonnera-t-elle ? Une alerte sans destinataire, sans action ou qui sonne tous les jours n'est pas une alerte, c'est un journal.

8. L'observabilité : lire une plateforme par ses traces

Labo : `Labo_11_DE.md`, étape 5.

L'observabilité d'une plateforme de données, c'est la capacité de répondre, sans ouvrir le code, à des questions sur son état : qu'est-ce qui est arrivé, quand, en quelle quantité, sous quelle forme, et qu'est-ce qui dépend de quoi. Le vocabulaire du métier en tire cinq piliers : la fraîcheur, le volume, la distribution, le schéma et le lignage. Les quatre premiers sont des dimensions de la section 3 ; le cinquième dit d'où vient une donnée et où elle va.

La plateforme de Boréal tient déjà ses traces, écrites cours après cours.

Trace	Écrite depuis	Ce qu'on y lit
`pipeline.chargements`	le Cours 03	chaque fichier chargé, son empreinte, son lot, ses lignes : la fraîcheur et le volume, livraison par livraison
`pipeline.executions`	le Cours 05	chaque passage d'une étape, sa durée, son état, ses lignes : ce que le pipeline a fait, et ce qu'il n'a pas fini
`pipeline.controles`	ce cours	chaque verdict de chaque règle, avec sa fenêtre : ce que la donnée valait quand elle est passée
`_fichier`, `_lot`, `_charge_le`	le Cours 03	pour chaque ligne de la `RAW`, le fichier d'où elle vient : le lignage au grain de la ligne

Le lignage a deux sens. Vers l'amont, `_fichier` ramène une ligne suspecte à son fichier, et le journal des chargements à son empreinte : les 609 commandes relivrées viennent toutes de `sources/erp/extraits/2023-03_bis/VBAK.csv`. Vers l'aval, le lignage dit qui sera touché si une source casse. Au grain de la table, il se déclare : c'est la ligne `consommateurs` de chaque contrat. Des outils le reconstituent de lui-même en lisant les exécutions ; le standard ouvert OpenLineage, par exemple, que la formation Architecture et Gouvernance reprend avec la gouvernance.

Une trace ne vaut que si elle se lit. Les trois tables vivent dans l'entrepôt, en SQL, comme le reste : une question sur l'état de la plateforme s'y pose avec les mêmes outils qu'une question sur les ventes. C'est le choix du Cours 05, qui a écrit les exécutions dans une table plutôt que seulement dans un fichier de journal, et ce cours en récolte le fruit.

© 2026 iAlexMG. Tous droits réservés.

9. Conduire un incident de données

Labo : `Labo_11_DE.md`, étape 11.

Un incident de données est un moment où une donnée livrée est fausse, absente ou en retard au point de nuire à quelqu'un. Il se conduit comme une panne de service, en cinq temps, et dans cet ordre.

Temps	Ce qu'on fait	Pour le micrologiciel 2.0
Détecter	un contrat rompt, ou un consommateur se plaint	la règle de schéma de la version 1.0.0, le 2025-05-01, sur quatre messages
Qualifier	mesurer l'étendue : quelles données, depuis quand, pour qui	les messages 2.0, de 4 le premier jour à 100 % du trafic en août ; tout lecteur de `temp_moteur_max`
Contenir	arrêter la propagation, prévenir les consommateurs	la tâche des contrats bloque `dbt_build` ; un mot aux équipes qui lisent la télémétrie
Corriger	réparer la cause, puis rejouer ce qui en dérive	le contrat en 2.0.0, les lecteurs réécrits avec les deux noms ; la fenêtre du Cours 06 rejoue les jours touchés
Apprendre	écrire le post-mortem, et changer ce qui l'a permis	les versions du micrologiciel annoncées au contrat avant leur diffusion

Le post-mortem est le document qui clôt l'incident. Il raconte ce qui s'est passé, heure par heure, mesure l'impact, remonte à la cause, et finit par des actions, chacune avec un responsable et une date. Il est sans blâme : il cherche ce qui, dans le système, a laissé passer l'erreur, et non qui l'a commise. Un post-mortem qui conclut « l'équipe du micrologiciel aurait dû prévenir » n'apprend rien ; celui qui conclut « rien n'obligeait à annoncer un changement de format, et aucune règle ne le vérifiait » change quelque chose.

L'idempotence construite depuis le Cours 03 prend ici toute sa valeur. Corriger, c'est rejouer : les jours touchés se rechargent par la fenêtre du Cours 06, et le journal des chargements garantit qu'un jour rejoué ne double rien. Un pipeline qu'on n'ose pas relancer transforme chaque incident en opération à cœur ouvert.

10. Les outils : ce que le marché ajoute

Le module du labo tient en deux cents lignes et fait voir chaque mécanisme. Les outils du marché font la même chose, avec plus de métriques, des rapports, des alertes branchées sur la messagerie, et un langage de règles à apprendre. D'après leurs documentations, consultées le 2026-09-23 :

© 2026 iAlexMG. Tous droits réservés.

Outil	Ce qu'il est	Sa forme	Bloquer ou avertir
Great Expectations (GX Core)	une bibliothèque Python ouverte, en version 1.23 ; Fivetran en assure la tenue depuis 2026	des suites d'attentes (`ExpectColumnValuesToBeBetween`, `ExpectTableRowCountToBeBetween`), lancées par des points de contrôle qui produisent des rapports	par les actions d'un point de contrôle
Soda Core	un moteur ouvert, dont la version 4, de janvier 2026, fait du contrat en YAML la forme par défaut	un fichier par table : `dataset`, `columns`, `checks` (`missing`, `invalid`, `duplicate`, `freshness`, `schema`…), des seuils `must_be...`	`level: warn` ou échec, par vérification
Les tests de dbt	les tests de données du projet d'Analytics Engineering	`unique`, `not_null`, `accepted_values`, `relationships`, et des tests écrits en SQL	`severity: warn` ou `error`, et des seuils

La parenté avec dbt est la plus directe, et elle est voulue. Un test dbt est une requête qui rend les lignes fautives, avec une sévérité : exactement une règle de contrat, posée plus loin dans la chaîne. Les contrats de ce cours vérifient ce qui arrive ; les tests d'Analytics Engineering vérifient ce qu'on en fait. Les deux sont nécessaires, parce qu'un `STAGING` peut casser une donnée qui était bonne à l'entrée, et qu'aucun test du `STAGING` ne sait qu'un mois n'est jamais arrivé.

Snowflake. Snowflake mesure la qualité par des fonctions de mesure de la donnée (*data metric functions*). Des fonctions système, `SNOWFLAKE.CORE.NULL_COUNT`, `DUPLICATE_COUNT`, `ROW_COUNT`, `FRESHNESS` ou `ACCEPTED_VALUES`, s'associent à une table ou à une colonne par `ALTER TABLE ... ADD DATA METRIC FUNCTION ... ON (colonne)`, avec au besoin une attente, `EXPECTATION pas_de_nul (VALUE = 0)`. Elles tournent selon `DATA_METRIC_SCHEDULE`, toutes les heures par défaut, et leurs résultats s'écrivent dans `SNOWFLAKE.LOCAL.DATA_QUALITY_MONITORING_RESULTS`. La fonction demande l'édition Enterprise, et les comptes d'essai n'y ont pas droit. D'après la documentation de Snowflake, consultée le 2026-09-23 ; la plateforme appartient à la formation Snowflake.

Databricks. Les pipelines déclaratifs de Databricks, les pipelines Lakeflow, autrefois Delta Live Tables, portent des attentes (*expectations*) sur chaque table qu'ils écrivent : `CONSTRAINT nom EXPECT (condition)` garde les lignes fautives et les compte, `ON VIOLATION DROP ROW` les retire, `ON VIOLATION FAIL UPDATE` arrête la mise à jour. En Python, les décorateurs `@dp.expect`, `@dp.expect_or_drop` et `@dp.expect_or_fail` font de même. Les comptes s'écrivent au journal d'événements du pipeline, sauf en cas d'échec. Trois degrés, les mêmes que ceux du labo, mais au grain de la ligne : retirer une ligne fautive est un choix que le labo refuse à la `RAW`. D'après la documentation de Databricks, consultée le 2026-09-23.

Recoupements — ce que ce cours partage avec ses voisines.

- Analytics Engineering : les tests dbt sont la version aval des contrats posés ici, avec la même forme, une requête et une sévérité. Leur écriture appartient au Cours 05 d'AE, leur place dans l'intégration continue à ses cours suivants.

© 2026 iAlexMG. Tous droits réservés.

- Architecture et Gouvernance : le contrat de ce cours est un fichier qui s'exécute. Archi en fait un engagement entre équipes, avec un propriétaire, un recours et une gouvernance des changements ; le lignage y devient un outil de gouvernance. Ici, on en voit la mécanique ; là-bas, on en décide.

Points clés à retenir

- La panne la plus grave ne plante pas : un mois absent, un fichier livré deux fois, un champ renommé passent au vert.
- Un contrat de données dit ce que le producteur promet et ce que le consommateur peut tenir pour vrai ; il vit dans un fichier versionné et s'exécute à chaque livraison.
- Une règle, c'est une métrique, un attendu et une sévérité, rangée dans une dimension : fraîcheur, volume, unicité, validité, complétude, intégrité, cohérence, schéma, distribution.
- Une règle de fraîcheur compare ce qui est là à ce qui était attendu, jamais seulement à ce qui est là.
- Bloquer, avertir ou suivre se décide par le coût de l'erreur en aval. Bloquer arrête ce qui se construit dessus, pas la donnée brute.
- Un changement qui casse fait monter le premier chiffre de la version du contrat, et se négocie avant d'arriver.
- Un contrôle de distribution se bâtit sur la vraie sortie : on le rejoue, on regarde chaque fausse alerte, et on la comprend avant de corriger.
- Chaque alerte se classe : vraie, fausse, manquée, ou vraie mais inutile. Une alerte qui sonne tous les jours est un journal.
- Les traces de la plateforme, chargements, exécutions et verdicts, se lisent en SQL comme le reste ; le lignage relie une ligne à son fichier.
- Un incident de données se détecte, se qualifie, se contient, se corrige et se raconte, sans blâme.

Exercices

Les énoncés et leur corrigé sont dans `Exercices_corriges_11_DE.md`. Faites le labo avant : plusieurs questions portent sur des chiffres que vous aurez mesurés vous-même.

© 2026 iAlexMG. Tous droits réservés.